

Behavioral Equivalence Cannot See Dormant Logic: A Pre-Registered Negative Result from a Five-Arm Comparison of PLC Command Gates on a Simulated PLC

WACE Technical Report TR-2026-01 (v1.1, revised 2026-09-22; v1.0 released 2026-09-21) — concept DOI 10.5281/zenodo.22868613

Dongkeol Bang — WACE Inc., Hwaseong, Republic of Korea (bangdk@wace.me)

Revised 2026-09-22 as a defensive publication (the company has decided not to file a patent on this work). Numbers are from official run 13 of the experiment (repository commit 8327000; its judgement counts are identical to run 10 at 4803a8a). The v1.0 numbers (run 6, commit 3e7351b) are kept in Table 1b and Table 2.

Revision note (v1.1). This version corrects two factual errors and two superseded statements in v1.0 and adds the results of the design revision that v1.0 announced as future work. The corrections are listed in full in the Errata at the end and in `ERRATA.md`. In short: (E1) four person-exposure false passes that v1.0 attributed to a removed interlock were in fact commands issued while the PLC was halted; (E2) v1.0 predicted that the other four would close when the rules were rewritten — they did not; (E3) v1.0 said the replica's permissive values were synchronised at least one second before judging — synchronisation happens at every judgement; (E4) v1.0's event-class account of the rule problem is superseded by measurement. **The main finding is unchanged and was measured again: adding the behavioral-equivalence check did not change what the gate lets through.**

Abstract

Command gates that pre-execute supervisory writes on a replica of the PLC program before they reach the controller are a known approach [1]. We ask whether adding a behavioral-equivalence check — confirming, from observed transitions, that the replica still matches the plant and blocking as undecidable otherwise — changes what the gate catches. In a pre-registered five-arm comparison on a simulated PLC (three self-authored ladder programs, 869 labelled commands, frozen catalogue and rule hashes, thirteen official attempts with all commits and records published), the arm with the equivalence check and the arm without it let through exactly the same hazardous commands issued while the replica was stale: 23 of 280 before the design revision described here and 19 of 278 after it, and after the revision the two arms are identical on every verdict count in Table 1 (p95 latency differs narrowly, 55.3 vs 55.0 ms). The four person-exposure false passes that remain are writes made while a timer preset had been changed in the plant program only — a dormant program difference that produces no behaviour until the timer runs, so neither look-ahead rules evaluated on the replica nor an observation-based freshness check can see it. Only a static program-signature check can, and on protocols without a program signature (Modbus) the gate cannot. Four further person-exposure false passes, which v1.0 attributed to a removed interlock, were in fact motor-start commands issued while the PLC was halted; a scan-counter heartbeat added in the revision refuses all 24 commands issued to a halted PLC. The same fail-closed gate blocked 0 of 94 commands that move the plant toward a safe state after the revision (2 before). Results are from a simulated PLC on one machine; no physical PLC comparison and no externally authored program are included. Our contribution is the evaluation discipline — pre-registration, run invalidation on judgement instability, no post-hoc relabelling, published denominators and excluded rows — shown alongside the numbers it prevented from being reported, including two wrong claims of our own v1.0 that this revision corrects.

1. Question

When supervisory systems (AI planners, MES, HMI) write directly to a PLC, a wrong command must be stopped in front of the equipment. Gates that hold a replica of the PLC program, run a candidate write a few scans ahead on that replica and block on an invariant violation are public prior art [1]. The premise is that the replica matches the plant. Plant programs drift — a maintenance bypass that is never restored, an online edit, a changed timer preset — and the replica then keeps judging on stale logic. A natural addition is a **behavioral-equivalence check**: confirm from observed transitions that the replica still matches the plant and, if it cannot be confirmed, block the write as undecidable. Our question is narrow: **does adding that check reduce hazardous false passes?** In this command set the answer is no, before and after the design revision, and the reason lies in the principle of the check itself.

Static pre-load verification of PLC code [3], the "ladder logic bomb" — logic that stays dormant until a trigger — and its proposed countermeasure of downloading the running logic and diffing it against a golden copy [4], and twin-based detection that lists "the twin models the correct behaviour" as its main limitation [5] are the closest prior work we could confirm in a first-pass survey of 33 sources; we found no controlled comparison that measures the marginal effect of an observation-based freshness check on a command gate.

2. Threat model — two adversaries that must not be conflated

(a) **An untrusted supervisory writer** — the AI planner, MES or HMI issuing the command. This is what the gate exists for, and every false-pass and false-block figure in this report is measured against this adversary.
(b) **PLC program drift of any origin** — maintenance not restored, online edits, malicious change — leaving the plant program different from the replica. This is not something the gate blocks; it is a *condition* under which the gate must decide whether its own replica can be trusted. An adversary who can write the PLC program cannot be stopped by any supervisory gate; that belongs to PLC write protection and change management (an IEC 62443 zone boundary). The contribution of this report is exactly: trying to detect (b) from behaviour is blind to dormant changes, so a static integrity check must come first.

3. Experimental design

Gate under test. An external device between the supervisory writer and the PLC. It synchronises the plant PLC's state (I/O, internal bits, timer elapsed values) into its replica at every judgement, executes the candidate write on the replica ahead of the plant, and blocks on an invariant violation. It holds no physical-process model, forwards no frames on the PLC's behalf, and proves by hash that judging leaves no side effect on the replica.

Design revision between v1.0 and v1.1. Two rules whose time windows looked back past the synchronisation point were rewritten as timer-level conditions (Section 5); a load-time refusal was added for such windows; each ladder gained a two-rung scan counter and the gate reads it repeatedly (up to five reads over 80 ms) before every judgement, refusing the write if the counter has not advanced (a halted PLC); permissive tags of start commands are re-read live after a passing look-ahead; a program-identity check and a signed allow-list of stop commands were added but **are not active on Modbus** and therefore not in the measured arms; the audit record format gained the corresponding fields.

Five arms (Table 1) add one component at a time. ARM3→ARM4 isolates the effect of internal-state synchronisation; ARM4→ARM5 isolates the effect of the behavioral-equivalence check. After the revision the three synchronising arms (ARM3–5) share the same scan-counter probe, so ARM4→ARM5 still differs only in the equivalence check.

Command set. 869 commands over three self-authored ladder programs (conveyor interlock, emergency stop, batch; 6, 19 and 21 rungs after the revision, two more each than in v1.0). Each command carries a replica-matches/replica-stale condition and a hazard-class label (person, product, record, kpi-only) assigned

before any run; the hashes of the catalogue, rule set and label file were frozen, and the revision re-bound them by tool, never by hand. No customer program was used.

Pre-registered rules. (1) An acceptance line (any block of a command that moves the plant toward a safe state for a reason other than replica mismatch disqualifies the run) and expected directions and values are written before execution. (2) No unchanged re-run; at most three attempts per cause (see Section 7 for how this rule was read in the revision). (3) If the three judging passes disagree on even one row, the run is invalid. (4) A row may be removed from the measured set only if the diagnosis predates the result, a mechanism is derived, and the exclusion is direction-neutral; inclusion and exclusion results are published side by side. (5) Numerators, denominators and excluded row IDs are published; what was not measured is never reported as zero. (6) Per-class counts are reported only as produced by one harness procedure — no hand tallies. (7, added before the revision's first run) The person-exposure class has an acceptance line of zero; a count above zero is reported as "person-exposure protection not claimed for this configuration".

Table 1. Five-arm results (run 13, after the revision; simulated PLC, three self-authored ladders, one machine)

Arm	Configuration	Hazardous cmds reaching PLC (of 284)	False pass (replica stale)	False block (replica matches, of 309)	Undecidable (stale / matches)	Safe-direction cmds blocked (disqualifying / baseline, of 94)	p95 ms
ARM1	No gate	284	246/284 (0.866)	0	—	—	0.7
ARM2	Static address/value rules only	133	95/284 (0.335)	128/309 (0.414)	0 / 0	47 / 0	1.5
ARM3	Look-ahead execution, output levels synchronised only, scan counter	62	28/282 (0.099)	5/309 (0.016)	0.092 / 0.371	0 / 0	—
ARM4	Look-ahead execution, internal state synchronised, scan counter	53	19/278 (0.068)	0/309 (0)	0.139 / 0.371	0 / 0	55.3
ARM5	ARM4 + behavioral- equivalence check	53	19/278 (0.068)	0/309 (0)	0.139 / 0.371	0 / 0	55.0

The ARM4/ARM5 denominator 278 is 284 hazardous commands minus 6 rows whose loaded state already violated a rule and were therefore not a judging target; the six row IDs are in the result file. ARM3's latency is not reported for this run because the machine was not idle while it was measured (the harness records this as non-exclusive and leaves the value empty). Latency is from one 16-core laptop; the scan-counter read before every judgement is the main added step since v1.0, and its share was not measured separately.

Table 1b. The same table in v1.0 (run 6, before the revision)

Arm	Hazardous cmds reaching PLC	False pass	False block	Undecidable (stale / matches)	Safe-direction blocked (disq. / baseline)	p95 ms
ARM1	284	246/284 (0.866)	0	—	—	1.0
ARM2	133	95/284 (0.335)	128/309 (0.414)	0 / 0	47 / 0	1.9
ARM3	66	32 (0.119)	13/309 (0.042)	0.060 / 0.386	0 / 2	36.4
ARM4	57	23/280 (0.082)	6/309 (0.019)	0.057 / 0.382	0 / 2	36.4
ARM5	57	23/280 (0.082)	6/309 (0.019)	0.070 / 0.382	0 / 2	35.5

4. Results

The equivalence check's marginal detection gain is zero — measured twice. In run 6, ARM4 and ARM5 false passes were identical at 23 of 280. The only difference in the two arms' verdict counts was four undecidable verdicts ("equivalence low") on four batch commands issued while the PLC was halted, which ARM4 had also blocked (by rule); that is the whole of the rise in undecidable verdicts under mismatch from 5.7% to 7.0% reported in v1.0. In run 13 the scan-counter probe refuses every command issued to a halted PLC first, in both arms, and ARM4 and ARM5 are identical on every verdict column of Table 1 (19 of 278 false passes, undecidable 13.9% / 37.1%, 0 false blocks); only p95 latency differs narrowly (55.3 vs 55.0 ms). In this command set, everything the equivalence check did was on rows where the PLC was halted, and a direct scan-counter check covers that class. Separately, on the run-6 tree, holding plant inputs for only 0.07–0.3 s blocked 16 of the 25 rows staged on the live-running plant as "replica freshness unconfirmed" (none under the adopted 1.2 s hold); this was not re-measured on the revised tree.

The 19 false passes by class (run 13). person 4, product 6, record 5, kpi-only 4. All 19 row IDs are among the 23 of run 6 — no new false pass appeared (checked by machine against a frozen list). **The four person-exposure rows are writes made while a horn duration or an arm dwell preset had been changed in the plant program only.** The replica runs its own preset, so any rule evaluated on the replica passes the command; rewriting the rule does not close them. v1.0 predicted that the rewrite would bring them to zero; **that prediction was wrong** (Errata E2). A program-signature check would see these differences, because the preset is a literal of the program; on Modbus no signature is exposed.

What the other four v1.0 person-exposure rows were. v1.0 described them as motor-start commands that passed while an interlock had been removed. **They were motor-start commands issued while the PLC was halted** (Errata E1). A halted simulated PLC reads its outputs as 0, so the replica, synchronised from those values, predicts no violation. The four rows pair the base program with a variant in both directions: in one the plant program had the interlock removed, in one the plant program had a changed horn preset, and in the other two the plant ran the base program while the replica held the variant. All four passed for the same reason — the PLC was halted. After the revision the scan-counter probe refuses all 24 commands issued to a halted PLC (0 reached the PLC, in all three synchronising arms). Hazardous commands sent to a *running* PLC whose program had the interlock removed are a separate group of 12 rows (8 person-exposure, 4 equipment); all 12 were blocked in run 6 and again in run 13 — the 8 person-exposure rows because the replica, which still holds the interlock, predicted the violation, and the 4 equipment rows for structural reasons (the write target was read-only or unmapped). None was blocked because the gate saw the removal. (One further command in that group is labelled safe; it was blocked too, which counts as a false block under a stale replica.)

Stop-command blocking, in the same paragraph. Of 94 commands that move the plant toward a safe state, 0 were blocked in run 13 (2 in run 6; both were horn-time stop commands whose blocking came from

a rule that the revision rewrote). Three judging passes were identical. This figure does not mean "the gate never blocks a stop": when the gate cannot confirm its replica it may block a stop command as undecidable, and the verdict does not consider whether a command reduces a violation. Hence category safety stops are wired around the gate as a deployment requirement (Section 8).

Baseline blocks and a missed expectation. Commands whose loaded state already violated a rule — blocked as undecidable regardless of the command — fell from 13 to 9. We pre-registered 7. The two extra rows are commands whose replica carries a changed arm-dwell preset: with the rule written as a timer level, the replica's timer can never reach the required 2 s, so the loaded state violates the rule at the synchronisation point. This is the intended behaviour (a replica that lacks the rule's premise must not pass commands), and it was missed by the expectation, not by the gate; it is reported as a missed expectation. Two of these rows are hazardous, which moves the false-pass denominator from 280 to 278.

Hazards confirmed unblockable: 3 kinds, 34 test commands (unchanged by the revision). Found by sweeping the 19 writable antecedent positions of the invariants and 2 command bits. Two are operations the ladder legitimately offers (remote reset, remote acknowledge), where the gate cannot distinguish a 1 written by the supervisory system from a 1 pressed by an operator. One is a 32-bit setpoint written as two words, where a single applied word leaves a value nobody commanded (of 30 legitimate changes, 17 were not applied and 4 were half-applied). Other positions have not been swept.

Features that did not act in this set. The live permissive re-read refused 0 commands (no row changes a permissive while a command is being judged — zero here reflects the set, not a missing feature). The program-identity check and the stop allow-list are not active on Modbus; in a test-only configuration that reads the simulated plant's program hash, a plant program that differs from the commissioned one is detected before any write is judged — the gate refuses to arm its allow-list, or latches "program changed" and refuses energising writes (the mechanism that would close the four remaining person-exposure rows). That configuration is not an arm and its numbers are not reported as results.

Run ledger. Thirteen official attempts: seven valid, four invalid, two aborted (Table 2). Invalid and aborted runs consume a number. Runs 7–9 failed on harness defects introduced by the revision; each was fixed together with a regression test that fails on the unfixed code, before the next attempt. Run 10 was valid; after its results were committed, a one-line test-fixture change altered the harness hash, the provenance check refused the recorded run as not belonging to the tree, and the run was measured again. Run 11 was aborted when the machine's Node.js runtime was upgraded during the run; run 12 was invalid because the machine went to sleep for 953 s and one row exceeded its look-ahead budget. Run 13 repeated run 10's judgement counts exactly.

Table 2. Run ledger

Run	Commit	Valid?	Change or cause	ARM5 false pass	Disqualifying / baseline stop blocks	Unmeasured rows
1	140b68d	valid · acceptance line [FAIL] · not adopted	original staging (halted-plant load)	22 (0.078)	2 / 2	0
2	71f890e	valid · superseded	live staging of 25 rows	23 (0.082)	0 / 4	0
3	5f20aa6	valid · superseded (loaded machine)	provenance guard fix	23 (0.082)	0 / 4	0
4	—	invalid (three judging passes disagreed on one row)	as runs 2–3	—	—	—
5	2c5c812	valid · superseded	4 rows unmeasured · denominators published	23 (0.082)	0 / 2	4
6	3e7351b	valid · adopted for v1.0	denominator carried into result file	23/280 (0.082)	0 / 2	4
7	174842e	aborted (stale cached rule file after the ladder hash changed)	design revision	—	—	—
8	8ee8368	invalid (model arms ARM3/ARM4 disagreed across passes)	cache fixed	—	—	—
9	0cf827e	invalid (model arms read the scan counter as a level)	plant run only for arms with the probe	—	—	—
10	4803a8a	valid · superseded (a later fixture changed the harness hash)	model arms given the probe	19/278 (0.068)	0 / 0	4
11	8327000	aborted (runtime upgraded on the machine mid-run)	fixture only	—	—	—
12	8327000	invalid (machine slept 953 s; one row exceeded its budget)	none	—	—	—
13	8327000	valid · adopted for v1.1	none (run under a sleep inhibitor on mains power)	19/278 (0.068)	0 / 0	4

5. Mechanism — why the equivalence check cannot see a changed preset

The behavioral-equivalence check confirms replica–plant agreement from **observed transitions**. A plant program whose timer preset has been changed moves **no tag differently** until that timer runs. With no difference in movement the check never enters suspicion, and the look-ahead judges the command on the replica's own preset, which says "no violation". This is not a defect of this implementation but the boundary of observation-based freshness checks as a class: a change that has not yet produced behaviour does not appear in behaviour. The four person-exposure false passes that remain after the revision are exactly this class.

What v1.0 got wrong about the mechanism. v1.0 used four motor-start rows as the example of this class and described them as commands passing a removed interlock that "moves no tag until a start command arrives", judged against "a replica value synchronised at least one second earlier". Both parts were wrong. Those rows were issued while the PLC was halted (a different class — see Section 4 — now refused by the scan counter), and the gate synchronises its replica at every judgement; the one-second figure is the sampling floor of the equivalence observer, not the age of the judging snapshot (Errata E1, E3). In this set, the hazardous commands to a running PLC with the interlock removed were all blocked — by the rules on the replica or on structural grounds.

What the rule rewrite actually fixed. v1.0 classified the invariant clauses by the events (edges) they contain and proposed refusing clauses whose events the PLC does not remember. Measurement during the revision showed that the false violations came instead from **window operators** (once, held) whose look-back

reached before the synchronisation point: re-seeding the replica mid-trajectory produced 20 and 203 scans of false violation in the two affected clauses, while edge operators were exact from the first scan after the seed. The two clauses were rewritten as timer-level conditions (for example, "while the horn is on, its timer's elapsed value is below 310 ms"), and twelve differential tests showed that the level form judges the same as the original from any seed, including negative controls that must — and do — show the original form failing when the replica is re-seeded mid-trajectory. A load-time refusal now rejects window operators longer than two scans in the position where they can only err toward passing; the committed rule set has zero refused clauses and one window left in the harmless position. v1.0's statement that "both stop blocks were class B; the four interlock-removal rows were class A" is superseded (Errata E4).

The closing means lie outside behavioral checking — and this direction is not a new recommendation.

Static comparison [4] and pre-load static verification [3] already point the same way; what this report adds is a measurement, in this class of gate, of why it is necessary. After the revision the gate has: (1) a program-identity check with a process-wide "program changed" latch that blocks every energising write outside a signed stop allow-list; it needs a PLC that exposes a program signature — the simulator's test-only source works, the physical-PLC sources (MELSEC MC, LS XGT) are not implemented because they require a physical PLC; (2) a live re-read of the permissive tags of start commands; (3) a scan-counter heartbeat. Only (3) acted in this command set. **On protocols with no program signature (Modbus), (1) does not exist**, the four remaining person-exposure rows stay invisible to the gate, and PLC write protection plus change management must be in place.

6. What the discipline caught — numbers that would have gone out, and the numbers reported

Rule	Event caught	Would have gone out	Reported
Three judging passes disagree → run invalid	Run 4, one row (the gate correctly suspected a halted plant)	Run 4 adopted as "valid"	Run 4 invalid; four rows marked unmeasured after root cause
Pre-registered acceptance line (stop blocks = 0)	Run 1: two stop commands blocked	"227 blocked" alone	Run 1 not adopted; harness cause fixed before re-run
Per-class counts only from one harness procedure	Lead's hand estimate: person 4	person 4	person 8 (run 6)
Numerators, denominators and excluded IDs published	Run 5 result file lacked the false-pass denominator	"8.2%" alone	23/280 plus four excluded IDs
No post-hoc relabelling	Registering four person rows as "unblockable" would cut the numerator 22→18 (v1.0 wording; these were the halted-PLC rows, see E1)	18	Kept in the numerator, published as a separate item
Asymmetric exclusion: three conditions plus two-way publication	Excluding four rows also removed the basis of the "zero stop blocks" claim (1 favourable, 7 unfavourable)	Exclusion effect only	Two-way table plus "this exclusion turns an invalid run into a valid one" stated
Harness provenance guard	Run 3 provenance-guard defect	Result file with wrong provenance	Fixed and re-run
Pre-registered values, not only directions	Baseline blocks: 7 expected, 9 measured	"As pre-registered"	9, with the cause, as a missed expectation
Our own prediction, checked	v1.0: four rule-shape rows close to zero after the rewrite	"Closed"	4 remain; prediction stated as wrong
Harness count before a claim goes out	A review stated "no row tests an interlock removed on a running PLC", and the sentence was drafted into outward wording	"Not measured"	12 such hazardous rows exist, all blocked — wording replaced before release
Judgement instability invalidates the whole run	Runs 8 and 9: model arms (not the product arm) disagreed across passes	Product-arm numbers from runs 8–9	Both runs invalid; harness fixed with regression tests first
Recorded run must match the tree	Run 10 valid, then a fixture changed the harness hash	Run 10 kept after the change	Re-measured as run 13 (identical judgement counts)

None of these principles is new — researcher degrees of freedom and the 21-word disclosure statement [6] [7], pre-coding of outcome variables and the garden of forking paths [8], catalogued benchmarking flaws [9], and the pitfalls of security evaluation (lab-only evaluation, label inaccuracy, inappropriate baselines) [10]; our two-way inclusion/exclusion table is requirement 5 of [6] as written. What we add is machine enforcement (a run is refused when the relaxation set intersects the disqualification-contributing rows; a record is not produced unless the headline figure rebuilds from its published components; a false pass outside the frozen list of run-6 IDs fails the gate) and the target (a safety gate on a frozen catalogue with a timing-sensitive harness). The discipline did not prevent v1.0 from publishing two wrong claims (Errata E1, E2); it did make them detectable, because the row labels and the pre-registered predictions were public.

7. Threats to validity

- **Construct.** Hazard-class labels were assigned before any run but by the team that built the gate; there is no independent-labeller agreement. The "3 kinds we confirmed unblockable" and the "6 hazards we deliberately let pass" are listed with class and reason in one table so that neither reads as denominator manipulation.
- **Internal.** Non-physical conditions created by the harness (loading onto a halted plant) were separated as four unmeasured rows with both inclusion and exclusion published. Run 3 was on a loaded machine; its latency is not used. In run 13, ARM3's latency is not reported for the same reason. During the revision the harness plant scans in real time while a command is judged for the three

synchronising arms; a row whose plant pump stalled for longer than the heartbeat window would be invalidated rather than counted — none was (the measure of stall is recorded per row).

- **Attempt cap.** The pre-registered cap "at most three attempts" was read as "three attempts per cause" before run 10 and after runs 7–9; this reading was recorded before run 10's result existed, and every attempt was preceded by a code or procedure change. Readers who hold the stricter reading should treat runs 10 and 13 as outside the pre-registration's attempt budget.
- **Lost row-level data.** The raw per-row records of run 6 were overwritten by a later run before they were copied. The aggregate result files of run 6 are intact, but the fall in false blocks from 6/309 to 0/309 (ARM4/ARM5) cannot be attributed row by row; the rule rewrite is the likely cause and is not claimed as verified.
- **External.** Simulated PLC assembled from product components; three self-authored ladders; one machine; zero externally authored programs. A halted simulated PLC clears its outputs; a physical PLC may hold them, depending on a parameter. The heartbeat window (80 ms) was sized to the simulator's 10 ms scan; on a physical PLC it must be at least twice the measured maximum scan time. The class in which an equivalence check should matter (forced bits; same program, values overwritten) is absent from this set, so "zero gain" is bounded to this set.
- **Scope of novelty.** In a first-pass survey of 33 sources we found no controlled comparison of the marginal effect of an observation-based freshness check; not finding one is not evidence that none exists.
- **Conclusion.** Exact counts on a fixed catalogue; no statistical test. Three identical judging passes are a reproduction on the same machine and tree, not statistical replication.
- **Generative-AI involvement.** A substantial part of the source code, experiment harness, design documents and this revision was written by an LLM coding agent under the author's direction, and the independent reviews and verifications were carried out by agents of the same model. The two v1.0 errors corrected here were found by harness counts and agent review, not by an external reader. Approval of the design, fixing of the pre-registration, run validity decisions, interpretation and every claim in this report were verified by and are the responsibility of the human author. "Independent judgement" here means agent runs that did not read each other's output, not human reviewers.

8. Deployment requirements and next steps

Requirements outside the gate. (1) Stop and de-energise actions are wired around the gate — the gate may block a stop command it cannot judge. (2) Remote-reset and remote-acknowledge bits are removed from supervisory authority. (3) 32-bit setpoints are written with a multi-register write (FC16). (4) When the PLC scan halts (STOP/PROGRAM mode, watchdog halt, online-edit pause) supervisory commands, including stop commands, are refused by design — a halted line whose PLC still scans is not refused; in run 13 all 24 commands issued to a halted PLC were refused. (5) The PLC program carries a two-rung scan counter placed where no conditional jump, subroutine or interrupt can skip it (first rung recommended), verified to advance in every operating mode; without it the gate starts with a warning that it cannot recognise a halted PLC. (6) Plant program changes go through change management that also updates the gate's replica; on Modbus the gate cannot notice a change that has not yet produced behaviour. Timer and counter internals must be exposed to communication, and what counts as a violation must be written for that plant.

Next experiments. (1) One physical PLC — above all, whether its program signature can be read (MELSEC MC, LS XGT); that decides whether the four remaining person-exposure rows can be closed on that protocol. Then the 25 live-staged rows, the 94 stop commands, forced bits, and the PLC's STOP output behaviour. (2) The forced-bit / value-overwrite scenario — the only class in which an equivalence check should matter. (3) At least one externally authored ladder. (4) Measurement of the suspicion-recovery interval (lower bound 8 s, no upper bound). (5) The sub-second input-hold curve on the revised tree.

Known limitations (mirrored in the repository's `KNOWN_LIMITATIONS.md`). Person-exposure false passes remain at 4 in run 13; all 4 are plant-only preset changes that only a program-signature check can see; on Modbus the gate cannot see them, so person-exposure protection is not claimed for the Modbus configuration. The halted-PLC refusal requires a scan-counter rung in the PLC program. A sub-second input change can hold the gate in suspicion with no time-based release (measured on the run-6 tree).

Errata (v1.0 → v1.1)

- **E1 — mechanism of four person-exposure false passes (factual error).** v1.0 (Abstract, Section 4, Section 5, `KNOWN_LIMITATIONS.md` item 1, Zenodo description): "motor-start commands issued after an interlock had been removed". Correct: motor-start commands issued while the PLC was halted (the simulated engine stopped, outputs read as 0). Only one of the four plant programs had the interlock removed; all four passed for the same reason. Hazardous commands to a running PLC whose interlock was removed (12 rows, 8 person-exposure) were all blocked in run 6 — by the replica's rules or on structural grounds, not by seeing the removal.
- **E2 — prediction (wrong).** v1.0 (Section 4): the four rule-shape person-exposure rows would close when rewritten as timer levels, "to be confirmed at zero in run 7". Measured: they remain (4 in runs 10 and 13). The difference exists only in the plant program's preset; the replica judges on its own preset; only a program-signature check can see it.
- **E3 — timing of the judged snapshot (factual error).** v1.0 (Abstract, Section 5): the permissive evaluated is "a replica value synchronised at least one second earlier". Correct: the replica is synchronised at every judgement; one second is the sampling floor of the equivalence observer.
- **E4 — event-class account (superseded).** v1.0 (Section 5, second paragraph): classes A/B/C of edge-carrying clauses, "an event the PLC does not remember cannot be used in an invariant", "both stop blocks were class B; the four interlock-removal rows were class A". Superseded by measurement: the false violations came from window operators looking back past the synchronisation point; two clauses were rewritten; zero clauses are refused.
- **Also changed in v1.1:** all headline numbers now come from run 13 (Table 1), with v1.0's run-6 table kept as Table 1b; the claim that the equivalence check "adds cost" is qualified (its only measurable effect in run 6 was on four halted-PLC rows, and in run 13 it has none).

References (sources whose text was opened and checked; verification levels are recorded in the survey file)

- [1] US 12,238,120 B1, "Embedded intrusion prevention system for industrial controllers", University of Alabama in Huntsville (inventors A. W. Werth, T. H. Morris), priority 2020-12-10, granted 2025-02-25.
- [2] KR 10-2004456 B1, "Apparatus and method for converting a PLC control program into structured data", UDMTEK Co., Ltd., priority 2016-08-12, granted 2019-07-30. (The gate in this report is designed not to practise the elements of its claims.)
- [3] S. McLaughlin, S. Zonouz, D. Pohly, P. McDaniel, "A Trusted Safety Verifier for Process Controller Code", NDSS 2014, DOI 10.14722/ndss.2014.23043.
- [4] N. Govil, A. Agrawal, N. O. Tippenhauer, "On Ladder Logic Bombs in Industrial Control Systems", ESORICS 2017 Workshops (CyberICPS 2017), LNCS, Springer, pp. 110–126, DOI 10.1007/978-3-319-72817-9_8; arXiv:1702.05241.
- [5] M. Eckhart, A. Ekelhart, "A Specification-based State Replication Approach for Digital Twins", ACM CPS-SPC '18, pp. 36–47, DOI 10.1145/3264888.3264892.
- [6] J. P. Simmons, L. D. Nelson, U. Simonsohn, "False-Positive Psychology: Undisclosed Flexibility in Data Collection and Analysis Allows Presenting Anything as Significant", *Psychological Science* 22(11), 2011, pp. 1359–1366, DOI 10.1177/0956797611417632.

- [7] J. P. Simmons, L. D. Nelson, U. Simonsohn, "A 21 Word Solution", *Dialogue* (SPSP newsletter) 26(2), Fall 2012.
- [8] A. Gelman, E. Loken, "The garden of forking paths: Why multiple comparisons can be a problem, even when there is no 'fishing expedition' or 'p-hacking' and the research hypothesis was posited ahead of time", unpublished manuscript, 2013-11-14.
- [9] E. van der Kouwe, G. Heiser, D. Andriesse, H. Bos, C. Giuffrida, "SoK: Benchmarking Flaws in Systems Security", *IEEE EuroS&P 2019*, pp. 310–325, DOI 10.1109/EuroSP2019.00031.
- [10] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, K. Rieck, "Dos and Don'ts of Machine Learning in Computer Security", *USENIX Security 2022*.